

GenoScale Supplementary

You

September 18, 2026

1 FAMR: Non-Linear Memory scaling

FAMR models a tool’s memory requirement as a power law in input size, fitted by least squares on the logarithm of requirement against the logarithm of input size over continuous sizes. Writing $x_i = \ln s_i$ and $y_i = \ln m_i$ for the n uncensored observations of a tool, the fit is

$$\ln \hat{m}(s) = \beta_0 + \beta \ln s, \quad \beta = \min\left(2, \max\left(0, \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}\right)\right), \quad (1)$$

with the intercept recomputed from the clamped slope, $\beta_0 = \bar{y} - \beta \bar{x}$, so the fitted line passes through the centroid of the data. Equivalently $\hat{m}(s) = e^{\beta_0} s^\beta$: the exponent is estimated per tool rather than assumed, and a size independent tool and a superlinear one are both representable. Log space is deliberate, because memory prediction error is multiplicative in practice, because the resulting margin is a ratio, which is the form the cost model in Eq. (3) needs, and because the fit cannot emit a negative requirement. The lower clamp at $\beta = 0$ is load bearing: a negative slope would recommend less memory for larger inputs, and it would also break the monotonicity assumption that lets a failure at a small input bound a prediction at a larger one (Eq. (5)). Spread is the residual standard deviation about the fit, with a floor,

$$\sigma = \max\left(\sigma_{\min}, \sqrt{\frac{1}{n-2} \sum_i (y_i - \beta_0 - \beta x_i)^2}\right), \quad \sigma_{\min} = 0.05, \quad (2)$$

so a tool observed once each at many distinct sizes is fully characterised, and a perfect fit through three points cannot claim that no headroom is needed. Headroom is then a cost balanced margin in units of σ , trading wasted memory seconds against the work a kill destroys. Treating $\ln m$ as normal with mean $\ln \hat{m}(s)$ and variance σ^2 , and dividing both costs by the predicted requirement and the run’s duration so that neither appears, the margin is

$$k^* = \arg \min_{0 \leq k \leq k_{\max}} \underbrace{\rho (e^{k\sigma} - 1)}_{\text{waste}} + \underbrace{(1 - \Phi(k)) e^{k\sigma} (\tau + \eta)}_{\text{retry}}, \quad (3)$$

where $\rho \in [0, 1]$ is cluster contention, $\tau = t_{\text{peak}}/t_{\text{total}}$ is the fraction of the run elapsed before the peak, η is the retry escalation, and Φ is the standard normal distribution function. Waste is priced by contention and a retry is not, so the margin widens as the cluster empties and tightens under load; the time to peak enters the cost of being wrong rather than the estimate itself, since a tool needs the same memory whenever it reaches it. Queries outside the fitted size range are priced in decades of extrapolation,

$$d = \max\left(0, \log_{10} \frac{s}{s_{\max}}, \log_{10} \frac{s_{\min}}{s}\right), \quad k = k^* + \lambda d, \quad (4)$$

which widens the margin and lowers reported confidence rather than asserting the same certainty at a size the fit has never seen.

2 FAMR: OOM Recovery

A run killed at its grant is recorded as a lower bound rather than a sample: it is excluded from Eq. (1) and used as a floor, taken from the grant rather than the sampled peak, since the fatal spike can fall between two samples. For the censored set $\mathcal{C}$, where g_i is the grant at which run i died, that floor propagates to every input size at least as large,

$$F(s) = \gamma \cdot \max \{g_i : i \in \mathcal{C}, s_i \leq s\}, \quad \gamma = 1.15, \quad (5)$$

escalated past the bound by γ because matching what died is still dying. The same rule covers a JVM live set read near its heap ceiling, which may report the ceiling rather than the requirement. Four safety rules then guard the loop. A circuit breaker is evaluated first: over a window $\mathcal{W}$ of the 20 most recent attempts,

$$\frac{|\{i \in \mathcal{W} : i \text{ failed}\}|}{|\mathcal{W}|} \geq \theta \implies M = 2 \max_{i \in \mathcal{W}} g_i, \quad \theta = \frac{1}{3}, \quad (6)$$

which grants generously and logs loudly, on the grounds that a tool failing a third of the time is not a model that needs nudging. Otherwise the recommendation is the fit, its margin, and the two floors, where C is the largest requirement proven necessary by a completed run,

$$M(s) = \max(\hat{m}(s) e^{k\sigma}, F(s), C), \quad (7)$$

subject to an asymmetric ratchet against the last grant g_{last} : a rise needs one observation, while a reduction is limited to

$$M \leftarrow \max(M, \delta g_{\text{last}}) \quad \text{whenever } c < c_{\min}, \quad \delta = 0.8, \quad c_{\min} = 5, \quad (8)$$

where c counts the most recent consecutive completed runs whose requirement fell below M and a censored run terminates the streak. None of these rules hold calibration state; each floor is a query over the measurements table, so nothing can drift out of sync with the data. Recovery itself is driven by typed pod level failure classification with deliberately asymmetric inference, counting container out of memory kills and JVM heap exhaustion apart because they name different ceilings, and escalating the grant that just failed over at most four attempts,

$$g_{a+1} = \phi_a g_a, \quad \phi = (1.5, 2.0, 3.0), \quad \text{retry declined when } g_{a+1} > g_{\max}^{\text{node}}, \quad (9)$$

so an unsatisfiable grant is reported with the figure it would have needed rather than silently rewritten to something placeable or held indefinitely. The escalation floor overrides both the model and an explicit resource override. Because one row is written per attempt, carrying granted memory beside observed peak, failure kind, attempt number, runtime, and the recommender that produced the grant, recovery is measurable directly: attempts to recovery per incident, recovered against terminal failures, retry time by failure kind, and out of memory rates split by runtime.